

EIFBENCH

1. Motivation

Current benchmarks don't reflect real world complexity

Most just evaluate single instruction + single constraint, or multi instruction but simple constraints.

Real-world tasks consist of multi instructions, multi constraints, and dependence between tasks.

Current LLMs struggle heavily in these settings.

2. Key Contributions

EIFBENCH (new benchmark)

Designed for multi instruction, multi constraint, multi task and multi modal

Total 8 task types, total 4 constraint types.

SegPO (Segment Policy Optimization)

Problem with prior method (GRPO):

- Uses global reward only

SegPO solution:

- Adds segment-level rewards per instruction

- Combines global advantage and instruction specific (segment) advantage

3. Eval Metrics

- Instruction Level Accuracy (ILA)

Did the model satisfy all constraints for each instruction?

- Constraint-Level Accuracy (CLA)

Measures individual constraint satisfaction

More fine-grained

Models often have decent CLA but very low ILA. they partially follow instructions but fail overall tasks.

This paper introduces EIFBENCH, a highly realistic benchmark for multi-instruction multi-constraint tasks, and SegPO, a training method that significantly improves performance by optimizing at the instruction level —revealing that current LLMs still struggle with real-world complexity.

Following Length Constraints in Instructions

1. Core problem

LLMs have a length bias

- They tend to generate longer answers
- Evaluations (and even humans) generally prefer longer responses

This is bad because longer $\neq$ better.

Models learn to game the system by writing more! They ignore instructions like 'Use $\leq$ 100 words, Be concise, etc'

Models like GPT-4 violate length constraints $\sim$ 50% of the time.

2. Key Idea

Instead of letting length be ambiguous, paper proposes to explicitly include length constraints in instructions and evaluate models on them.

3. Built

New benchmarks! They modify existing benchmarks

- AlpacaEval $\rightarrow$ AlpacaEval-LI
- MT-Bench $\rightarrow$ MT-Bench-LI

How?

They take a normal prompt and add a length constraint.

4. What they measure

1. Violation Rate (Vlt%)

% of responses that break the length limit

2. Win Rate

If the model follows length $\rightarrow$ compare quality vs baseline
violates length $\rightarrow$ automatic loss

5. Findings

State-of-the-art models FAIL at length control

- GPT-4 variants violate constraints 40-50%
- Even strong models can't reliably control output length.

6. Solution:

LIFT = Length-Instruction Fine-Tuning

Train models to respect length constraints explicitly

They take normal training data:

Prompt -> Good Answers vs Bad Answers

And modify it:

Case 1: Longer answer originally wins

They create two training examples.

1. Add a loose length limit -> both answers valid, original winner stays winner.
2. Add a *tight* length limit -> longer answer now violates and becomes loser.

This teaches the method that sometimes being shorter is more important than being detailed.

They use:

- DPO (direct preference optimization)
- Model learns when to prefer short vs long answers

7. Results

Example (Llama models):

- Violation rate
 - Before: ~65%
 - After (LIFT): ~7%

Huge improvement in following length constraints!

The problem is training + evaluation. Models are trained on signals that favor length, so they learn longer = better.

8. Proposal

Train models to treat constraints (like length) as first-class requirements

Critical for token limits (APIs), UI (chat boxes, summaries), etc.

Without this, models can waste tokens and ignore instructions.

MATH

1. Evaluation (AlpacaEval-LI / MT-Bench-LI)

How they measure length-following.

They take a prompt, and add a constraint 'answer in $\leq$ MAX_LEN words'

MAX_LEN is chosen and run 3 strong models:

- GPT-4 Turbo
- Claude Opus

- Mistral Large

and take the MINIMUM of the length of all their outputs $\min(x,y,z)$ as MAX_LEN

It guarantees a feasible constraint and ensures the baseline will always satisfy it.

Metric 1: Violation Rate (Vlt%)

$Vlt = (\# \text{ num violations} / \# \text{ total samples}) \times 100$

What counts as violation? If $L_{\text{output}} > \text{MAX_LEN}$

10 prompts: 4 outputs too long $\rightarrow$ violations, so 40%.

Metric 2: Win Rate (LI Win%)

This is a pairwise comparison.

First we compare the model vs the baseline.

Each prompt has a baseline response (always valid length), and the model response (may violate).

We apply a hard constraint rule.

1. Case A: Model violates length, model loses automatically.
2. Case B: Model satisfies length, then use a judge model. Win if $\text{quality}(\text{model}) > \text{quality}(\text{baseline})$

$\text{Win \%} = (\# \text{ wins} / \# \text{ total comparisons}) \times 100$

2. LIFT (training method)

How they teach models to follow length

They start with preference data: $D = (x_i, y_i^w, y_i^l)$ where x_i is the prompt, y_i^w is the winning response, y_i^l is the losing response.

They do a filter step, where they remove pairs where lengths are too similar

$$|\text{len}(y_i^w) - \text{len}(y_i^l)| < T$$

with $T = 10$.

This is because if lengths are similar, we get no useful length signal.

They add length instructions, x'_i which introduces an 'answer in $\leq L$ words' + x_i instruction.

They flip preferences based on length constraints. It creates NEW training examples by adding a length constraint.

- Case 1: Winner is longer. $\text{len}(y_i^w) > \text{len}(y_i^l)$

They create two training examples.

Constraint both satisfy $L = \text{len of winner} + T$, then both responses valid

and original preference stays, winner still wins. Pick a limit so both responses are valid. When there's no pressure from length, pick the better answer.

Constraint in between

Pick a limit that splits them (so L between both length sizes). Now: the original winner is invalid so the old loser becomes the winner. Even if a response is higher quality, it loses if it breaks the constraint.

We need both cases because if you only trained on one case, the model would get confused. Shorter is better.. but only sometimes! If we only train on the inbetween constraint, the shorter lower quality answer will be better, but the model might incorrectly generalize that shorter answers are better. The rule depends on context (length constraint).

Best answer is then highest quality if constraint valid, invalid answers lose if constraint violated.

Final Training Dataset: D' = augmented data with length constraints.

Train on the union of D and D' .

Training objective: DPO (Direct Preference Optimization). How much more the model prefers winner over loser (first log is how much the model likes the winning answer, second log is how much it likes the losing answer.) make winner more likely than loser.

D is standard. better answer, more informative, etc.

D' (length-augmented) so preferences become, sometimes long answer wins sometimes short answer (because long violates length constraint).

We mix both datasets to learn what is good content and constrained data to change what 'better' means. This forces the model to learn that without constraint, prefer the high-quality longer answers, and with constraint prefer valid-length answers even if worse quality.

LIFEBENCH

1. Motivation

LLMs are bad at following length instructions.

Models don't reliably control output length. They often fail on long outputs and don't reach their claimed max length.

2. Proposal

LIFEBENCH (new benchmark)

A benchmark to test: Can LLMs follow length instructions across different tasks, languages, and lengths?

Dataset:

- 10800 examples
 - 4 task types
 - QA
 - Summarization
 - Reasoning
 - Creative writing
 - 2 languages
 - English + Chinese
 - Length range:
 - 16 to 8192 words
- 3 types of length control
equal to -> exact length
at most -> max limit
at least -> min length

3. Evaluation

Length Deviation (LD)

How far off was the output length?

$$LD = \frac{L_{output} - L_{constraint}}{L_{constraint}}$$

Positive -> too long, negative -> too short

Length Scores (LS)

Converts deviation into a 0-100 score. 100 = perfect match.

Fixed problems like averaging errors cancelling out, extreme outliers.

4. Results

LLMs struggle with length control. Most models score < 60/100 on exact-length tasks. Only a few (reasoning models) do better.

Short vs long. Good at short outputs, fail badly at long outputs (>= 4066

words) No model consistently performs well at long lengths.

Claimed vs actual max length. Models don't reach their advertised limits.

Many fail far below their max token capacity.

5. Why Models Fail

No "length awareness"

Models can't accurately track how long their output is. They underestimate short outputs, overestimate long ones. Some even claim they met the length without actually doing so.

Input length hurts performance.

Longer input -> worse length control. Summarizing a long input is the worst performing.

Lazy strategies

Models cheat when tasks are hard, they stop early, refuse to continue, give partial answers. Happens more on longer lengths.

6. Other Findings

Task matters

Worst: summarization, Best: creative writing.

Language matters

Worse performance in Chinese. Tend to over-generate.

MATH

Start with 360 base tasks (QA, summarization, reasoning, creative)

Then apply 3 control types (= , > , <). 10 length values:

16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192

So total

$$360 \times 3 \times 10 = 10,800$$

test cases. Each test case = (input, instruction + length constraint)

For each instance, the model generates

- L_output: number of words it actually produced
We already know:
- L_constraint: required length

Core metric: Length Deviation (LD)

This is the raw error. (formula above) It's a relative error (percentage-like).

Cases are:

- Perfect match: $L_{output} = L_{constraint} \implies LD = 0$
 - Too short (under generation): $LD < 0$
Constraint = 100, Output = 80, do $LD = -0.2$, 20% too short.
 - Too long (over-generation)
 $LD > 0$. Output = 100, constraint 100, $LD = 0.2$. 20% too long.
- There is a problem with LD. If you average over many samples, -0.2 and 0.2 cancel out and the average is misleading. Huge errors will dominate averages. So they introduce a second metric: Length Score (LS)

Length Score (LS)

They transform LD into a bounded score (0-100)

$LS = \text{piecewise}(100 e^{k_1 \times LD}, \text{ if } LD < 0, 100 e^{-k_2 \times LD} \text{ if } LD \geq 0)$

where $k_1 = 5$ (penalize short outputs harder), $k_2 = 2$ (penalize long outputs less)

Why this shape? Under-gen is worse, so penalize more aggressively.

- Perfect: $LD = 0$, $LS = 100$
- Slightly wrong, $LD = \pm 0.1$, LS will be high = 80-90
- Very wrong, LS close to 0.